

Authentication System for Images

Krishna Taneja
1020231339

1. Problem Definition

Over the past few years, the rise of AI-generated images and sophisticated editing tools has been significantly evident; gravely corrupting the authenticity of digital photographs we consume every day. For sectors like journalism and legal evidence, determining the authenticity is becoming increasingly crucial as deepfakes and manipulated photographs are used in fake news and misinformation. Traditional approaches rely on metadata (EXIF tags) which can be easily forged, or human judgment which fails against sophisticated AI-generated content.

This project focuses on implementation of a subset of the problem: a cryptographic image authentication system based on the principles of C2PA (Coalition for Content Provenance and Authenticity) standard. The system provides proof of image authenticity and detects any tampering.

2. Threat Model

Threat Actors

- Actors who create realistic synthetic images using generative AI tools.
- Malicious editors (using Photoshop/GIMP) to tamper with authentic photographs
- Actors who remove authentication signatures to hide evidence of tampering
- Social media users stripping metadata before redistributing altered images

Adversary Capabilities

- Generate entirely synthetic images using AI
- Modify pixel data of authentic images (edits like color changes, object removal, cropping)
- Remove signature files and meta data from files

Trust vs Verifiability

Current Model: Images are trusted based solely on the reputation of the source with no actual cryptographic proof. EXIF metadata can be easily forged. Thus, detection relies mostly on human judgment.

Proposed Model: Trust to be shifted to mathematical proof via concept of digital signatures. Anyone can verify authenticity using the manufacturer's public key.

Trust assumptions:

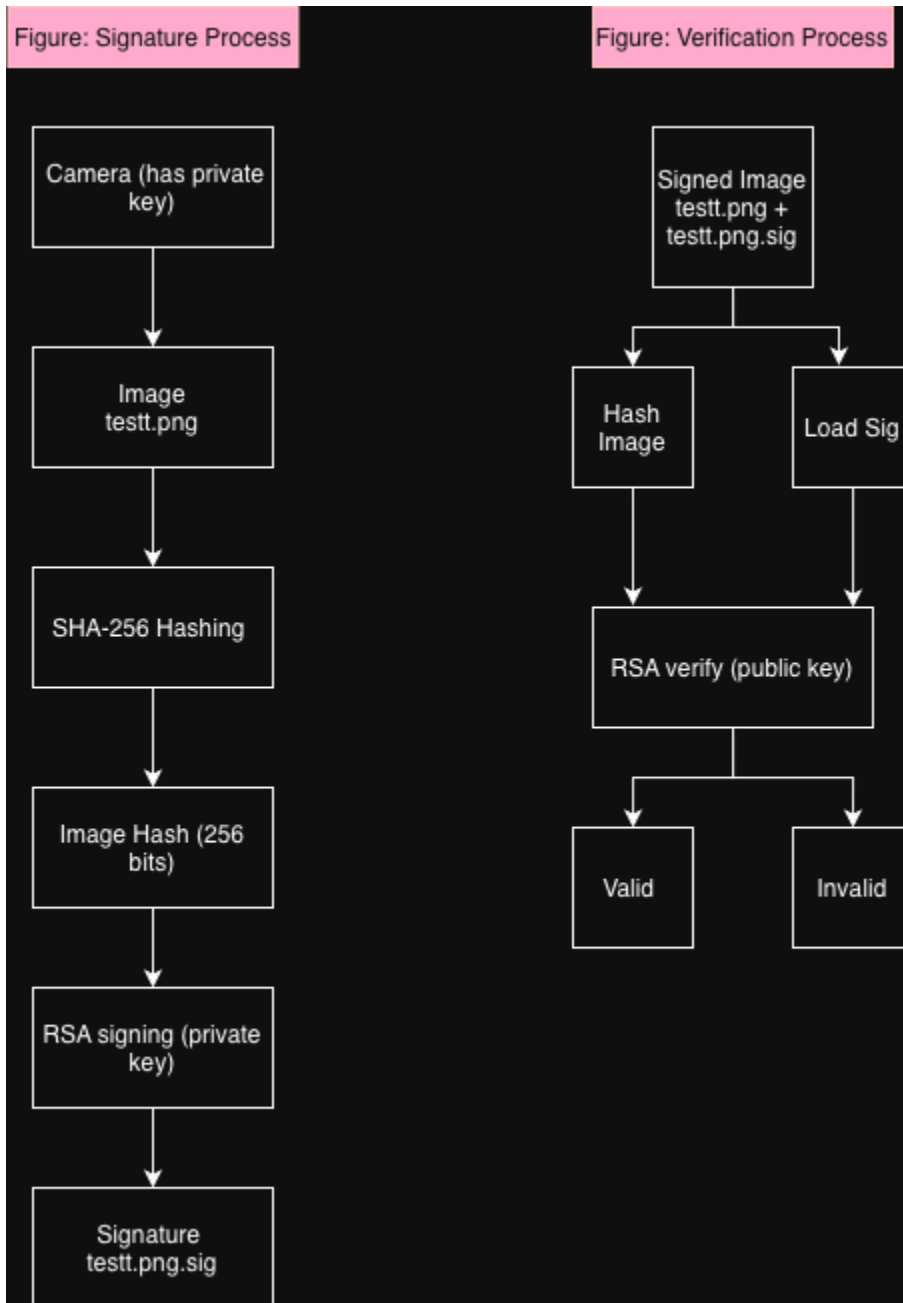
- Private keys are secured in the hardware by the manufacturer of the camera
- Public keys distributed via HTTPS, RSA-2048 and SHA-256 remain secure

Attack Scenarios and Mitigations

- AI-Generated Fake: Adversary creates synthetic image using AI tools. System detects missing signature and returns "NO SIGNATURE FOUND."
- Image Tampering: Adversary modifies authentic photo (changes pixels, removes objects). Any edit changes the hash, causing verification to fail with "TAMPERED."
- Signature Removal: Adversary deletes signature file to hide tampering. Image flagged as "UNVERIFIED" - cannot be trusted.
- Signature Replay: Adversary copies signature from one image to another. Verification fails because signature is cryptographically bound to original hash.

This proposed system combines RSA-2048 digital signatures with SHA-256 hashing to ensure authenticity of images.

3. System Architecture Diagram



4. Interaction of Mechanisms

SHA-256 Hashing: a fixed size cryptographic fingerprint is created. Detecting tampering is made to be easily detectable as even a single-bit change produces a completely different output. However, hashing alone can not prove authenticity, since just about anyone can compute a hash.

RSA Digital Signatures: helps prove that only the camera could have produced the signature. The data is signed by the private key, and the public key verifies it. But signing big, multi-megabyte image files directly can be computationally expensive and infeasible.

Interaction: Thus, these mechanisms must work together to mitigate individual limitations. SHA-256 reduces arbitrary-sized images to fixed 256-bit hashes, which directly make RSA signing more efficient. The signature then cryptographically binds this hash to the camera's identity.

Public Key Infrastructure: Similar to SSL certificate authorities, camera manufacturers can publish the camera's public key using HTTPS-secured websites. Then, verifiers can download public keys once, and then can authenticate an unlimited amount of images, even without network access.

End-to-End Security: From capture (camera signs with private key) through storage (image + signature stored together) to verification (anyone validates with public key), each step is protected via cryptography.

Thus, this architecture ensures security across the entire lifecycle.

- Confidentiality: the private key never leaves the secure hardware. It is only the public key that's known to the public.
- Integrity Preservation: signature travels with the image through untrusted channels
- Layered Cryptography Mechanisms: image can only be verified when both signature and hashing matching are confirmed

5. Security Analysis

Implementation Testing: Developed a Python prototype and tested against four different attack scenarios:

Test 1: authentic image uploaded: Original image signed and verified.

Result: prints: AUTHENTIC with matching hash. System correctly validates legitimate images.

Test 2: AI-generated image: Synthetic and generated image without a signature is submitted.

Result: prints: NO SIGNATURE FOUND. System identifies unverified content.

Test 3 : tampered image: signed image is modified (even if a single pixel changed), signature file preserved.

Result: prints: TAMPERED. SHA-256 hash differs; RSA verification fails mathematically.

Test 4: replay of signature: Valid signature is copied to a different image.

Result: prints: TAMPERED. Any signature is cryptographically bound to an original hash; it cannot validate a different image.

Security Properties Achieved:

- Attackers cannot forge signatures without private keys (RSA hardness)
- cannot modify images without detection (SHA-256 collision resistance)
- cannot transfer signatures between images (cryptographic binding)
- cannot repudiate signed images (non-repudiation).

6. Limitations

- Requires camera hardware integration for deployment.
- No revocation mechanism for compromised keys that are currently in implementation.
- Timestamps can still be forged before signing. Thus, the system proves integrity but not temporal authenticity
- Sophisticated attacks preserving file-level hash while changing semantic meaning are not addressed.
- Depends on secure public key distribution channels.

Output Screen for an original picture that is tampered with + random unsigned photo

```
krishataneja@Krishas-MacBook-Air infosec_project % python3 authenticate.py sign testt.png
Image signed: testt.png.sig
Hash: c4e870d358d64ce677e958c156d0e3fb...
Signature: b52d0e74817c992c7694ade096cdce7f...
krishataneja@Krishas-MacBook-Air infosec_project % python3 authenticate.py verify testt.png
AUTHENTIC
Image hash: c4e870d358d64ce677e958c156d0e3fb...
krishataneja@Krishas-MacBook-Air infosec_project % python3 tamper2.py
Created tampered image with signature intact
krishataneja@Krishas-MacBook-Air infosec_project % cp testt.png.sig testt_tampered.png.sig
krishataneja@Krishas-MacBook-Air infosec_project % python3 authenticate.py verify testt_tampered.png
TAMPERED
krishataneja@Krishas-MacBook-Air infosec_project % python3 authenticate.py verify random.png
NO SIGNATURE FOUND
krishataneja@Krishas-MacBook-Air infosec_project %
```